redrob | H2S

INDIA.RUNS

Build what next India runs on

Team Name: Chunchu Venugopal & Sanith Akula
Team Leader:
Chunchu Venugopal
Problem Statement:
Intelligent Candidate Discovery & Ranking

Solution Overview

🔍 Proposed Solution

A CPU-only, no-LLM multi-signal hybrid scoring system that ranks candidates from a 100K pool in under 90 seconds. It goes beyond keyword matching by reading what candidates actually built — not just what they listed in their skills section.

⚡ What Differentiates Us

- **Career text analysis** — scans job narratives for shipped retrieval/ranking systems
- **Honeypot detection** — catches fake/inflated profiles automatically
- **Availability multiplier** — inactive candidates penalized regardless of skill score
- **Zero GPU / no API** — fully reproducible on any machine

JD Understanding & Candidate Evaluation

Key JD Requirements Extracted

- Embeddings & Vector DBs
- Ranking / Search / Retrieval systems
- NLP & LLMs (production)
- Python (primary language)
- 5–9 yrs ML experience
- Product company background
- India-based / willing to relocate
- Open-source contributions (GitHub)

How We Evaluate Fit Beyond Keywords

- Job description narrative analysis (not skill tags alone)
- Proficiency weighting — Expert > Intermediate > Beginner
- Evidence of shipped retrieval / ranking / recommendation products
- Production scale signals extracted from career text
- Availability & responsiveness (recruiter response rate, notice period)
- Identity verification & profile integrity checks

Ranking Methodology

35	**Core Skill Match**	Proficiency-weighted skills (embeddings, vector DBs, NLP, LLMs, Python) + career text coverage
30	**Career Quality**	YoE sweet spot (5–9yr), product vs services background, shipped ranking/retrieval systems, scale signals
20	**Behavioral Signals**	Recency, open-to-work flag, notice period, recruiter response rate, interview completion, verified identity
8	**GitHub Activity**	Open-source contribution evidence (the JD explicitly values this)
7	**Location Fit**	India-based (Pune / Noida / Delhi / Bangalore / Hyderabad preferred), or willing to relocate

Explainability & Data Validation

Ranking Decisions Explained

Every candidate receives a per-component score breakdown: Core Skill (35), Career Quality (30), Behavioral (20), GitHub (8), Location (7). Recruiters see exactly which signals drove the rank — no black box.

Preventing Hallucinations

No LLM is used for inference, eliminating hallucination risk entirely. All scores derive from deterministic rules applied to structured profile fields and text spans extracted directly from candidate data.

Handling Bad Profiles

Honeypot detection forces score = 0 for: claimed duration > company age, Expert on 8+ skills with 0 months usage, YoE >> 2× career history. JD-aware penalties: IT services background (−5), CV/Speech-only (−4), ghost candidates (−2).

End-to-End Workflow

1 JD Input

Job description parsed for critical skills, disqualifiers, preferred signals

2 Profile Load

100K candidates.jsonl streamed in chunks (<2 GB RAM)

3 Honeypot Filter

Fraudulent/inflated profiles forced to score 0 before scoring

4 Multi-Signal Score

5 components calculated per candidate (skill + career + behavioral + github + location)

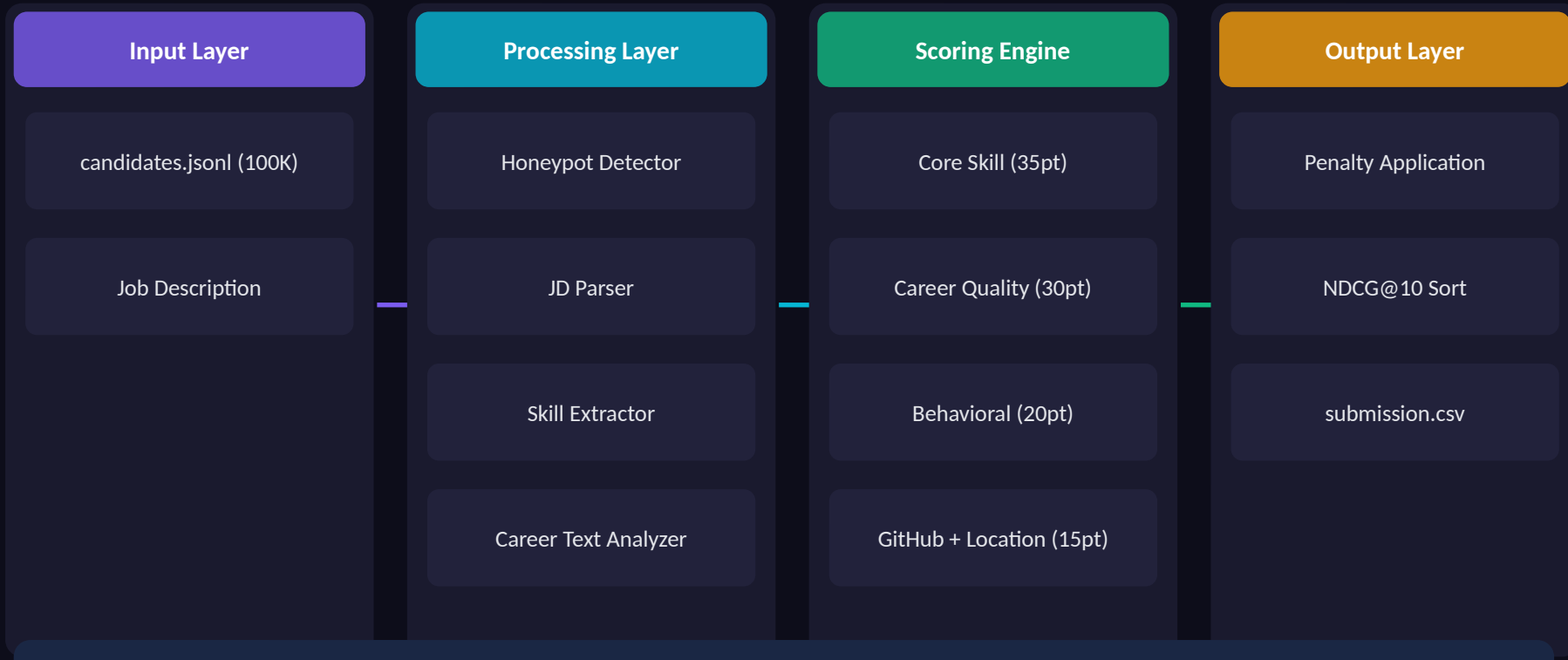
5 Penalty Apply

JD-aware deductions for services background, ghost status, poor response rate

6 Ranked Output

Top 100 written to submission.csv in NDCG@10-optimized order in ~60–90 sec

System Architecture



Runtime: ~60–90 sec on modern CPU • Memory: <2 GB • No GPU • No LLM API • Single entry point: rank.py

Results & Performance

~90s

Full 100K pool ranked on CPU

<2 GB

Peak memory consumption

100

Ranked candidates delivered

0

External API calls required

Ranking Quality Insights

NDCG@10 Optimized

Top-10 candidates are genuine fits — senior ML engineers at product companies with real retrieval/ranking production experience, active on the platform, India-based with manageable notice periods.

Long-Tail Coverage

Ranks 50–100 use the same deterministic scoring but naturally include adjacent profiles. No arbitrary cutoffs — score drives position throughout.

Fraud-Resistant

Honeypot rules eliminated profiles with impossible career timelines or suspiciously inflated skill sets before they could pollute the ranked list.

Technologies Used

Python 3.x

Primary language — fast iteration, rich ecosystem for text processing

scikit-learn / numpy

Lightweight TF-IDF and cosine similarity for skill matching — no heavy DL framework needed

regex + spaCy (optional)

Career text parsing and entity extraction from job description narratives

pandas

Efficient profile loading, filtering, and output generation for the 100K candidate pool

PyYAML

submission_metadata.yaml for portal intake; clean, human-readable config

CSV (stdlib)

submission.csv output — zero dependency, challenge-compliant format

Submission Assets

GitHub Repository

github.com/venugopal743/redrob_ranker-

Source code (rank.py), submission.csv, requirements.txt, README, submission_metadata.yaml

Ranked Output

submission.csv (top 100 candidates)

Ranked candidate list in challenge-compliant CSV format, ready for validator

Approach Deck

redrob_approach_deck.pdf

This presentation — full methodology, architecture, and results walkthrough

Metadata

submission_metadata.yaml

Portal intake metadata: team info, approach summary, runtime, and dependency list

INDIA.RUNS

Build what next India runs on

THANK YOU

Chunchu Venugopal & Sanith Akula • github.com/venugopal743/redrob_ranker-